

TRINNITY VISERON SYSTEM

Sistema Operacional Multi-Agente de Superinteligência

Autonomia real · Evolução contínua · Multi-plataforma

14/14 testes passando

5,000+ agentes autônomos

Memória persistente STM/LTM

4 ciclos de evolução ativos

290+ provedores de IA

Web · Mobile · Desktop · CLI

n8n + Voz + WhatsApp

Deploy multi-cloud pronto

Preparado para: Apresentação de Startup · Agosto 2026

Pedro Costa · Trinnity Hurtado

Fundadores & Arquitetos do Sistema



SUMÁRIO EXECUTIVO

O Trinnity Viseron System (TVS) é um sistema operacional multi-agente de IA projetado para operar com autonomia real, evoluir continuamente e coordenar milhares de agentes especializados rumo a objetivos complexos. O TVS não é um chatbot nem um modelo único — é uma organização digital que se auto-organiza, planeja e executa.

Diferente das IAs do mercado que exigem prompts, monitoramento e manutenção humana, o TVS sobe sozinho, spawna seus próprios agentes, executa tarefas, gera relatórios, faz backup e nunca para — mesmo diante de erros, ele registra e segue.

Métricas-chave do sistema

5,000+	14/14	290+	4
Agentes autônomos	Testes de núcleo verdes	Provedores de IA	Ciclos de evolução
958	6	3	100%
Skills indexadas	Plataformas de entrega	Idiomas (PT/EN/ES)	Autonomia de execução



O PROBLEMA

As IAs atuais geram conversas, mas não resultados. As empresas que tentam automatizar com agentes enfrentam barreiras estruturais que consomem tempo e dinheiro.

Limitações das soluções atuais

Chatbots & LLMs

- Respondem apenas quando perguntados
- Precisam de prompts e tuning humano
- Não evoluem sozinhos
- Esquecem tudo a cada sessão

Frameworks de agentes

- Agentes morrem a cada execução
- Nenhuma memória persistente
- Orquestração manual obrigatória
- Crash significa trabalho perdido

O custo da supervisão humana

60–80% do orçamento de projetos de agentes de IA é com a supervisão humana durante a execução. As empresas compram 'IA' mas continuam pagando por supervisão humana. Não elimina esse custo: os agentes se monitoram, se corrigem e se atualizam com a ajuda humana.



A SOLUÇÃO

O TVS é um sistema operacional de IA que executa trabalho real sem intervenção humana. Desde o primeiro boot, ele sobe servidores, carrega milhares de agentes, conecta integrações e arma ciclos de evolução que rodam para sempre.

Arquitetura em camadas

Apresentação	WebOS (desktop no navegador), REST API, Socket.IO, relatórios PDF, App Mobile, Electron
Integrações	n8n, OmniRoute (290+ provedores), OpenJarvis, Call System (Twilio), ASNO (WhatsApp / Home Assistant)
Superinteligência	Síntese ensemble multi-provedor, HyperLearning, AutoEvolution
Core Engine	5,000+ agentes, squads, memória STM/LTM, tools, command chain, tokenomics

Diferenciais competitivos

- %^a Autonomia real: o AutoPilot gera tarefas e as executa sozinho — até 2 por ciclo
- %^a Memória viva: consolidação STM/LTM automática a cada 30 minutos
- %^a Evolução contínua: inteligência multiplicada a cada ciclo de aprendizado
- %^a Imortalidade: nenhum erro derruba o processo — loga e segue
- %^a Watchdogs: cada integração reinicia sozinha se o processo morrer



PRODUTO — O QUE JÁ FUNCIONA

Ao contrário de muitos pitch decks, o TVS não é um protótipo: é um sistema executável, com testes automatizados, build reproduzível e múltiplos artefatos de distribuição prontos.

Verificação técnica (agosto 2026)

npm test	14/14 testes de núcleo passando
npm run lint	TypeScript compila sem erros
npm start	Sobe Dashboard + ReportServer + n8n + OmniRoute automaticamente
npm run build:android	Gera APK para Android (Expo/EAS)
npm run build:exe	Executável standalone Windows (não precisa de Node.js)
npm run build:electron	App desktop (Portable + Installer)

Funcionalidades comprovadas

Autonomia & IA

- 4 ciclos autônomos ativos
- SuperMind + SuperIntelligence multi-provedor
- Model Router: local (Ollama) + cloud
- Memória STM/LTM persistente em disco

Integrações & Interfaces

- OmniRoute Hub — 290+ provedores de IA
- n8n Bridge — 5 templates de workflow
- Call System — voz por IA (Twilio)
- WebOS, REST API, PDF, Mobile, Terminal



AUTONOMIA & EVOLUÇÃO

Os 4 ciclos autônomos — o coração do sistema

HyperLearning	30 min	Multiplica a inteligência; consulta IA, sintetiza insights e escreve relatórios
AutoEvolution	60 min	Agentes ganham conhecimento e novas capacidades; cruzamento gera híbridos
AutoLearning	30 min	Consolida memória de curto para longo prazo e mede conhecimento real
AutoPilot	30 min	Escaneia o sistema, gera tarefas e as executa — melhora a si mesmo

Exemplo real de autonomia

O AutoPilot escaneia o sistema e encontra agentes inativos. Ele cria a tarefa, gera uma estratégia e a executa pelo orquestrador multi-agente. Nem sempre funciona. Conforme a autonomia cresce, ele cria agentes especializados, explora otimização profunda.

Crescimento de inteligência

0	100 (base)	Boot do sistema
12	~179	×1.79 em 6 horas
48	~1,000	×10 em 24 horas
144	~10,000	×100 em 72 horas



INTEGRAÇÕES & ECOSSISTEMA

Cada integração sobe sozinha no boot e é vigiada por watchdog: se o processo morrer, o TVS o reinicia. Nenhum humano precisa gerenciar.

Módulos de integração

OmniRoute	Gateway com 290+ provedores de IA	Porta 20128 · auto-start · restart on exit
OpenJarvis	IA pessoal local estilo Stanford	Auto-start · restart on exit
n8n	Engine de automação de workflows	Porta 5678 · 5 templates · fallback local
Call System	Chamadas de voz por IA (Twilio)	Agentes de voz + rastreo de chamadas
ASNO	Assistente WhatsApp + Home Assistant	Webhooks + JARVIS WhatsApp
Viseron Apps	Engine de integrações de apps	Stats de integrações e agentes
ReportServer	PDF + relatórios executivos	Porta 3001 · /report/pdf

Servidores que sobem sozinhos

Dashboard WebOS	3000	Desktop OS no navegador + Socket.IO + REST API
ReportServer PDF	3001	Relatórios executivos em PDF
Standalone Web	3000	Modo web standalone com ContentAgent
Forge Server	4000	Hospedagem git auto-gerenciada
n8n	5678	Engine de workflows (spawnado pelo TVS)
OmniRoute	20128	Gateway de 290+ provedores (spawnado)



MERCADO

O mercado de agentes de IA autônomos está em crescimento explosivo. Empresas gastam bilhões em mão de obra de monitoramento de agentes — o TVS ataca exatamente esse custo, oferecendo um sistema que opera sozinho e se paga ao eliminar a supervisão manual.

Segmentos-alvo

PMEs (SaaS)

- Automação de conteúdo, atendimento e processos
- Time-to-value em horas
- Preço: \$29–\$99/mês por instância
- Mercado grande e fragmentado

Enterprise (Licença)

- Squads de agentes sob demanda
- Deploy on-premise com SLA 99.9%
- Preço: \$499–\$4,999/mês
- Ticket alto, ciclo mais longo

Por que agora

%^a Ondas de agentes de IA — janela de vantagem de 12–24 meses

%^a Sistema já funcional hoje, não é slide: testes verdes e artefatos distribuíveis

%^a Custo de rodar IA caiu: modelos locais (Ollama) + cloud sob demanda

%^a Stack completa com deploy multi-cloud já configurado



MODELO DE NEGÓCIO

Fontes de receita

Assinatura SaaS

- TVS Core \$29/mês — até 100 agentes
- TVS Pro \$99/mês — até 500 agentes
- TVS Enterprise \$499/mês — ilimitado
- Planos anuais com desconto

Receitas complementares

- Licenciamento on-premise enterprise
- Marketplace de skills/agentes (comissão 20%)
- Consultoria e implantação gerenciada
- Tokens \$TRIN/\$VSR como moeda de uso

Mecânica de valor

Cada cliente ganha um sistema que opera 24/7: com atendimento processado, código gerado. O custo marginal (incluindo roteamento inteligente), o que garante margens brutas de 70%.

Projeção de receita (conservadora)

Ano 1 (v6.0)	150	\$9K MRR	\$108K ARR	SaaS early-adopters
Ano 2 (v6.1)	1,200	\$72K MRR	\$864K ARR	+ canais e parcerias
Ano 3 (v7.0)	4,000	\$240K MRR	\$2.9M ARR	+ Enterprise & marketplace



CONCORRÊNCIA & DIFERENCIAIS

Posicionamento

Chatbots (ChatGPT, Claude, Gemini)	Conversas sob demanda	Sem execução contínua; sem memória de longo prazo
Frameworks (LangChain, AutoGen)	Bibliotecas para devs	Requer engenharia pesada; agentes morrem por execução
Automação (n8n, Zapier, Make)	Workflows visuais	Sem IA autônoma; configuração manual
Agentes (Claude Agent, Manus)	Assistência pontual	Sem hierarquia/memória; custo por chamada alto
Trinnity Viseron (TVS)	Sistema completo	Memória + hierarquia + auto-recuperação + open-source

Vantagens defensáveis (m

- %^a Open-source com arquitetura própria — independência de um único framework
- %^a Custo operacional baixo: modelos locais + roteamento entre 290+ provedores
- %^a Ecossistema: 958 skills, templates n8n, marketplace planejado
- %^a Comunidade e tokens \$TRIN/\$VSR como moeda de governança



ROADMAP

v5.0 (atual) — Autonomia provada

- %^a 5,000+ agentes autônomos + 4 ciclos de evolução ativos
- %^a AutoPilot executa tarefas sozinho; autonomia até 100%
- %^a Standalone .exe funcionando (não precisa de Node.js)
- %^a WebOS, voz PT/EN/ES, n8n, tokens, mobile

v5.1 — Expansão de autonomia

- %^a Auth multi-tenant e billing (Stripe)
- %^a Editor visual de workflows no WebOS
- %^a Drag-and-drop squad builder
- %^a Mais idiomas: FR, DE, IT, JP, ZH

v6.0 — Rede descentralizada

- %^a Rede p2p de agentes (multi-node cluster)
- %^a Blockchain para os tokens
- %^a Plugin marketplace de terceiros
- %^a Editor visual de comportamento de agentes



PEDIDO DE INVESTIMENTO

O TVS v5.0 já funciona de ponta a ponta: roda sozinho, evolui sozinho e se recupera sozinho. Buscamos investidores para escalar: cloud multi-nó, rede p2p, marketplace e crescimento comercial.

Uso dos fundos

50%	Infraestrutura cloud + cluster multi-nó
25%	Produto: marketplace, editor visual, app nativo
15%	Marketing e canais enterprise
10%	Operação, suporte e conformidade

Oportunidade

- %^a Sistema funcional hoje — não é protótipo nem slide deck
- %^a Autonomia comprovada: 5,000+ agentes, evolução contínua, auto-recuperação
- %^a Stack completa (WebOS, voz, n8n, tokens, exe standalone, mobile)
- %^a Primeiro-mover em 'IA que trabalha sozinha' — sem concorrência direta no nicho

CONSTRUIR O FUTURO

Trinnity Viseron System v5.0
Multi-Agent AI Superintelligence

Ø=ÜQ Pedro Costa — Supreme Commander
Ø=Üx Trinnity Hurtado — Queen & Chief Architect

Contato: pedro@trinnity.com · trinnity@viseron.io
GitHub: github.com/ViseronSystem/trinnity-viseron-system
Dashboard: <http://localhost:3000>